

NOTE MÉTHODOLOGIQUE

Détection de fraude sur le Mobile Money

Démarche, variables, modèle et résultats — pipeline complet des volets A (fraude), B (budget) et C (classement client)

Équipe — Nextdata

Date — 20 juillet 2026

AUC-PR (holdout final) — 0,91

Périmètre — 1 627 622 transactions · 175 689 clients

01 Le problème traité

Le paiement mobile BAMIS traite des dizaines de milliers d'opérations par jour. Le vrai risque n'est pas le fraudeur qui dépasse un seuil — c'est celui qui le contourne : fractionnement en petits montants, comptes mules qui font transiter l'argent, multiplication d'opérations pour rester sous les radars. La difficulté centrale n'est pas de détecter la fraude, c'est de ne pas signaler à tort les clients honnêtes (marchands, agents, salariés) dont le comportement ressemble à de la fraude sans en être.

La solution répond à trois questions liées : (A) cette transaction est-elle frauduleuse, ce compte est-il une mule ? (B) où en est ce client par rapport au seuil de son service, quel seuil devrait-il avoir ? (C) quel est le risque et la valeur de ce client, comment le traiter ?

02 Préparation des données

Le dictionnaire de données du cahier des charges annonce 23 colonnes. Un audit complet du fichier réel (1 627 757 lignes) montre que chaque ligne contient en réalité 26 champs. Cause : les colonnes de date exportent une fraction sub-seconde après une virgule non protégée par des guillemets (ex. `15:05:44,421000000`), lue par tout parseur CSV standard comme une colonne supplémentaire, ce qui décale silencieusement toutes les colonnes suivantes. Sans correction, un montant peut être lu comme un identifiant — une corruption invisible tant que personne ne compare les valeurs à leur sens attendu.

Le mapping réel des colonnes a été reconstruit par inférence statistique sur le fichier complet (`configs/schema_map.yaml`), avec un niveau de confiance explicite par colonne. Deux autres constats : (1) aucune colonne cible de fraude n'existe dans ce fichier — le jury garde la vérité terrain de côté (voir section 4) ; (2) 63 lignes à date aberrante et 72 lignes à montant aberrant (jusqu'à 10^{60} MRU) ont été mises en quarantaine avant tout calcul.

03 Variables construites

Le pipeline sépare `ingestion/` (lire le fichier tel qu'il est réellement structuré), `preprocessing/` (nettoyer, sans encore de signal métier) et `feature_engineering/` (variables prédictives). Quatre catégories de variables, toutes calculées de façon strictement causale (uniquement à partir de transactions antérieures, jamais la transaction courante ni le futur) :

- **Seuil** (`threshold_features.py`) : ratio montant/seuil du service, distance au seuil, cumul journalier/seuil journalier — 0,78 % des transactions au-dessus du seuil unitaire, 2,16 % dans la zone 80-100 % (fractionnement à surveiller).
- **Comportemental** (`behavioral_features.py`) : écart au montant médian et au maximum habituel du client (historique propre, fenêtre expanding strictement antérieure), nouveau bénéficiaire, délai depuis la dernière opération. Principal levier contre les faux positifs : un compte à 3 400 opérations/24h s'est révélé, une fois comparé à son propre historique, être un agent légitime et non une fraude.
- **Vélocité** (`velocity_features.py`) : opérations et montant cumulé sur 1h/24h/7j par client, glissant et causal — capture rafales et fractionnement temporel.

- **Réseau léger** (`network_features.py`) : montant reçu vs envoyé et son ratio (signal mule), délai depuis la dernière réception, nombre d'expéditeurs/destinataires distincts. Approximation rapide en attendant le module graphe (niveau 3) — 3,71 % des transactions présentent un profil "réception puis réexpédition rapide".

04 Règles métier et classement client

Sept règles (`rules/business_rules.py` , niveau 1), chacune correspondant à un type de fraude du cahier des charges (R1/R2 : dépassement de seuil unitaire/journalier ; R3 : fractionnement ; R4 : rafale (>80 op./h) ; R5 : nouveau bénéficiaire + montant record (F-03) ; R6 : nocturne hors norme (F-01) ; R7 : profil mule (C-02)). Un score de suspicion agrège le nombre de règles déclenchées.

Rôle double : au-delà du filet de sécurité explicable, ce module fabrique le **pseudo-label d'entraînement** du modèle ML — une transaction est un exemple de fraude si ≥ 2 règles se déclenchent (0,47 % des transactions, cohérent avec le " ~ 1 %" du cahier des charges).

Paramètres calibrés contre les vraies données, pas choisis arbitrairement : le seuil de rafale (R4) fixé à 80 op./h après mesure du 99^e percentile réel (86, une première tentative à 15 flaguait 26 % des transactions à tort) ; la fenêtre "nuit" (R6) recalibrée de 22h-6h à 0h-7h (22h/23h sont en réalité des heures d'activité normale) ; la bande de ratio reçu/envoyé du signal mule (R7) resserrée de [0,8-1,2] à [0,9-1,1] (la bande large matchait 28 % des transactions — pas discriminant, c'est le délai depuis la dernière réception qui porte le signal utile).

Limite assumée : R7 reste la moins sélective des 7 règles (3,71 %) — recevoir puis payer sous 30 min est un comportement légitime courant (marchand, agent). Le signal mule fin est délégué au module graphe (niveau 3), qui distingue un pass-through isolé d'un pattern répété.

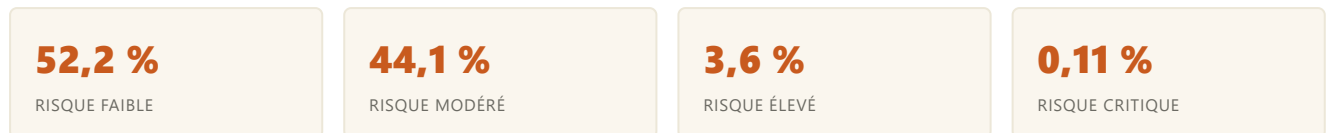
Gestion de budget (volet B) : pour chaque couple client×service, les montants sont cumulés sur la journée et le mois, **tous canaux confondus** (`budget/budget_engine.py`) — aucune colonne canal n'entre dans le regroupement, c'est précisément ce qui empêche le contournement C-08 (un client qui atteint son seuil sur l'appli puis continue chez un agent voit son compteur continuer, pas repartir de zéro). Vérifié par test unitaire (`tests/test_budget_engine.py`). Les seuils sont toujours lus depuis `configs/thresholds/seuils_services.csv` , jamais codés en dur, pour rester modifiables par le jury. Une alerte est émise à 50 %, 80 %, 95 % et 100 % du seuil (`budget/alert_engine.py`) — sur les 946 850 lignes client×service ×jour, 2 509 ont atteint 100 % du seuil journalier.

Bonus — seuil personnalisé par classe de risque (lien explicite demandé par le cahier des charges entre volet B et volet C) : `budget/threshold_recommender.py` traduit l'action de la matrice de traitement (volet C, ci-dessous) en un multiplicateur appliqué au seuil officiel — de $\times 1,5$ (bon client, risque faible) à $\times 0$ (gel/investigation), valeurs lues depuis `configs/config.yaml` , hypothèses commerciales à valider par BAMIS, pas des règles officielles. Résultat : `outputs/reports/recommandations_seuils.csv` , 67 107 recommandations.

Classement client (volet C) : deux notes indépendantes (risque/valeur, 0-1000) pour **175 689 clients** (population étendue le 2026-07-20 des 40 866 émetteurs à tous les comptes vus dans le fichier — un compte qui ne fait QUE recevoir de nombreux expéditeurs différents est le profil collecteur/fan-in (C-

03) que le cahier demande de détecter, absent auparavant), avec exactement les 5 sous-critères et poids du cahier des charges (`configs/config.yaml`).

Méthode de normalisation **corrigée deux fois après vérification empirique** : un premier essai en rang percentile écrasait tous les clients à égalité vers le 50^e percentile (la plupart des indicateurs sont concentrés à zéro), si bien qu'aucun client ne descendait sous 367/1000 et que "Risque faible" restait vide — corrigé par un min-max borné pour les indicateurs concentrés à zéro et un rang percentile conservé pour les indicateurs continus (montants, ancienneté). Un second ajustement a remplacé la moyenne par le **maximum** au sein de chaque sous-critère : un client extrême sur un seul indicateur ne doit pas être dilué par ses indicateurs normaux.



Bug trouvé et corrigé le 2026-07-20 : deux colonnes de `network_features.py` (`nb_expéditeurs_distincts_past` / `nb_destinataires_distincts_past`, calculées via `merge_asof`) donnaient des valeurs fausses à l'échelle complète (un cas vérifié montrait "544 expéditeurs" pour un client qui en a réellement 13, confirmé deux fois par comptage direct) — cause exacte non identifiée dans le temps disponible. Colonnes retirées du score client, remplacées par un comptage direct revérifié.

Limite assumée : ces colonnes faisaient aussi partie des 24 variables d'entrée du modèle CatBoost déjà entraîné — non réentraîné avec la version corrigée faute de temps, impact non mesuré mais probablement limité (2 variables sur 24).

Autres limites : "Profil"/"Ancienneté" se limitent à la durée de la relation (aucune donnée KYC) ; "Rôle dans un réseau" n'inclut pas "téléphone partagé" (impossible à vérifier sans identifiant indépendant du téléphone). **Explicabilité** : chaque score étant une somme pondérée de 5 sous-scores nommés, l'explication consiste à les trier par contribution et les traduire en phrases (`scoring/explainability.py`).

05 Modèle de Machine Learning choisi

Trois modèles comparés sur le même pseudo-label :

MODÈLE	AUC-PR (CV 3 REPLIS)	AUC-PR HOLDOUT FINAL
Baseline (score de règles)	0,42	—
Régression logistique	0,31 (±0,21)	—
CatBoost (retenu)	0,76 (±0,20)	0,91 (554 positifs / 212 222 ex.)

La régression logistique fait moins bien que la baseline elle-même : confirme que le problème n'est pas linéairement séparable dans l'espace des 24 variables retenues.

Pourquoi CatBoost : pour des données tabulaires, le gradient boosting est l'état de l'art. CatBoost a été préféré pour (1) son *ordered boosting*, qui limite le surapprentissage sur une cible imparfaite — pertinent

puisque notre cible est un pseudo-label, pas une vérité terrain — et (2) peu de réglage d'hyperparamètres nécessaire, un avantage réel avec un budget de 2 jours. LightGBM envisagé mais non testé (score déjà net) ; XGBoost et le stacking écartés (plus de surface pour une fuite de données, gain marginal attendu).

Déséquilibre de classes : aucun sur-échantillonnage — CatBoost gère le déséquilibre nativement via sa fonction de perte, un ré-échantillonnage aurait ajouté un risque de fuite temporelle. L'AUC-PR (section 7) reste informative malgré le déséquilibre extrême (~0,5 % de positifs). Modèle sauvegardé dans `models/catboost_v1.cbm`, réutilisable via `modeling/predict.py`.

06 Méthode de validation

Découpage temporel glissant (walk-forward), jamais aléatoire (`validation/temporal_split.py`) : 3 replis glissants (entraînement sur une fenêtre, validation sur la période immédiatement suivante) ; **embargo de 7 jours** entre train et validation, pour empêcher qu'une variable calculée avec retard ne fuite d'un côté à l'autre ; **holdout final** = 5 % les plus récents (recalibré de 15 % à 5 % le 2026-07-19 après vérification que même à 5 % du temps, 554 cas de pseudo-fraude restent disponibles — suffisant pour un AUC-PR stable, tout en préservant 87 % des données pour l'entraînement).

Contrôles automatiques anti-fuite (`validation/leakage_checks.py`, qui font échouer le pipeline plutôt qu'avertir silencieusement) : absence de chevauchement de dates, respect de l'embargo, absence de doublon de code transaction entre train et validation — **PASS sur les 3 replis**, vérifié par test unitaire (`tests/test_leakage_checks.py`).

Anti-fuite au niveau des variables : les flags des 7 règles métier sont explicitement exclus des variables d'entrée du modèle (`feature_store.py`), alors même que le pseudo-label en est dérivé — les inclure aurait permis au modèle de recopier trivialement la formule du label.

07 Résultats obtenus

0,91 AUC-PR HOLDOUT FINAL	319 TRANSACTIONS BLOQUÉES	64,5 M MRU PROTÉGÉS
-------------------------------------	-------------------------------------	-------------------------------

AUC-PR holdout final : 0,91 (554 vrais positifs / 212 222 exemples), contre 0,42 baseline et 0,31 régression logistique.

Précision/rappel (`outputs/reports/training_metrics.json`) :

SEUIL	PRÉCISION	RAPPEL	ALERTES
0,1	18,7 %	100 %	2 961
0,3	23,7 %	99,6 %	2 329

Rappel quasi total même à seuil bas — cohérent avec l'objectif de ne rater aucun cas suspect, quitte à trier les faux positifs ensuite (volet C).

Impact chiffré (score de suspicion $\geq 3/7$, simulation de politique de blocage) : 319 transactions bloquées, ~64,5 millions MRU protégés.

Module graphe (bonus) : 163 259 comptes et 1 363 346 transactions en graphe, 24 637 circuits fermés détectés, meilleur candidat mule à 259 pass-through rapides sur 273 transactions (95 %).

08 Limites de la solution

- **Aucune vérité terrain de fraude** : le 0,91 mesure la capacité à retrouver notre propre pseudo-label, pas la vraie fraude gardée par le jury — contrainte du problème, assumée.
- **Bug dans 2 des 24 variables du modèle** (section 4) — corrigé pour les scores client, modèle non réentraîné faute de temps, impact non mesuré.
- **Colonnes à confiance faible** (`CHANNEL_TYPE` , `SETTLEMENT_STATUS` ..., 38-51 % de vides irréguliers) volontairement non utilisées — conséquence directe : C-08 (changement de canal) n'est pas détecté.
- **Patterns graphe non couverts** : chaînes de rebond 3+ sauts (C-05) et fractionnement multi-comptes (C-10) — trop coûteux à calculer correctement sur 163 259 comptes dans le temps disponible.
- **Aucune donnée KYC** : "Profil"/"Ancienneté" limités à la durée de la relation.
- **Seuil mensuel (volet B) estimé** à 30× le seuil journalier (aucun seuil mensuel officiel fourni) — hypothèse documentée, à remplacer si un vrai seuil est communiqué.
- **Modèle non calibré** au sens probabiliste strict — sans conséquence sur l'AUC-PR (qui ne dépend que du classement), à signaler si le score doit être interprété comme une vraie probabilité.

09 Améliorations possibles

- Reconnecter le module graphe complet (vrai score mule, appartenance à un circuit fermé) dans `scoring/customer_scoring.py` , qui utilise aujourd'hui une approximation.
- Réentraîner le modèle après correction des 2 variables buguées.
- Détecter les chaînes de rebond (C-05) et le fractionnement multi-comptes (C-10) avec un budget de calcul plus large.
- Seuils adaptatifs par classe de risque : partiellement fait (bonus `budget/threshold_recommender.py` , `outputs/reports/recommandations_seuils.csv`) — reste à valider les multiplicateurs avec la direction des risques BAMIS.
- Calibration probabiliste formelle (`modeling/calibration.py` , non implémenté) si le score doit être une vraie probabilité.
- Détection du changement de canal (C-08) si une colonne canal fiable est un jour fournie.